Informe Ejecutivo: Análisis del Mercado Automotriz

Fecha: 2026-05-02
Analista: [Tu Nombre]
Archivo: `TP_Final`

---

1. Plan de Exploración Estructurado

Para este análisis se aplicó un flujo de trabajo de cuatro etapas:

1. **Diagnóstico:** Limpieza y tratamiento de nulos.
2. **Ingeniería de Características:** Normalización y creación de variables binarias.
3. **Análisis Exploratorio (EDA):** Identificación de correlaciones clave.
4. **Validación:** Pruebas de hipótesis para confirmar hallazgos.

In [2]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

import warnings
warnings.filterwarnings('ignore', category=DeprecationWarning)

# Nombres oficiales en español
nombres_columnas = [
    'riesgo_seguro', 'perdidas_normalizadas', 'marca', 'tipo_combustible'
    'aspiracion', 'numero_puertas', 'estilo_carroceria', 'traccion',
    'ubicacion_motor', 'distancia_ejes', 'largo', 'ancho', 'alto',
    'peso_vacio', 'tipo_motor', 'numero_cilindros', 'tamano_motor',
    'sistema_combustible', 'diametro_cilindro', 'carrera_piston',
    'relacion_compresion', 'caballos_fuerza', 'rpm_maximas',
    'consumo_ciudad_mpg', 'consumo_autopista_mpg', 'precio'
]

# Cargar el archivo sin cabecera y asignarle los nombres
df = pd.read_csv('automoviles.csv', header=None, names=nombres_columnas)

# Verificamos que se cargó bien
print(df.dtypes)

#Veo cuantos datos estoy manejando
print(f"\nDimensiones del dataset: {df.shape}")
```

```
riesgo_seguro              int64
perdidas_normalizadas        str
marca                        str
tipo_combustible             str
aspiracion                   str
numero_puertas               str
estilo_carroceria            str
traccion                     str
ubicacion_motor              str
distancia_ejes           float64
largo                    float64
ancho                    float64
alto                     float64
peso_vacio                 int64
tipo_motor                   str
numero_cilindros             str
tamano_motor               int64
sistema_combustible          str
diametro_cilindro            str
carrera_piston               str
relacion_compresion      float64
caballos_fuerza              str
rpm_maximas                  str
consumo_ciudad_mpg         int64
consumo_autopista_mpg      int64
precio                       str
dtype: object

Dimensiones del dataset: (205, 26)
```

1. **Diagnóstico:** Limpieza y tratamiento de nulos.

In [3]:
```
#LIMPIEZA DE DATOS
#1. Identificar el tipo de datos de cada columna para entender qué tipo d
# 1. Conversión a numérico (Excelente uso de coerce)
cols_numericas = ['perdidas_normalizadas', 'diametro_cilindro', 'carrera_
                  'caballos_fuerza', 'rpm_maximas', 'precio']

for col in cols_numericas:
    df[col] = pd.to_numeric(df[col], errors='coerce')

# 2. Imputación por la media
# Tip: He añadido 'precio' a la lista de exclusión para no inventar preci
for col in ['perdidas_normalizadas', 'diametro_cilindro', 'carrera_piston
    df[col] = df[col].fillna(df[col].mean())

# 3. Imputación por la moda (puertas)
moda_puertas = df['numero_puertas'].value_counts().idxmax()
df['numero_puertas'] = df['numero_puertas'].fillna(moda_puertas)

# 4. Eliminación de filas sin precio
df.dropna(subset=['precio'], axis=0, inplace=True)

# 5. Reinicio el indice
# Al borrar filas con dropna, el índice queda con huecos (ej: 1, 2, 4...)
# Esto causa errores en visualizaciones y modelos más adelante.
df.reset_index(drop=True, inplace=True)

# 6. Verificacion de los nulos
```

```python
# Una simple línea para confirmar que no quedaron valores vacíos.
print("--- Conteo de valores nulos finales ---")
print(df.isnull().sum())

# 7. TIPADO CORRECTO
# Columnas como 'caballos_fuerza' no suelen tener decimales,
# convertirlas a int ayuda a la legibilidad.
df['caballos_fuerza'] = df['caballos_fuerza'].astype('int')
```

```
--- Conteo de valores nulos finales ---
riesgo_seguro            0
perdidas_normalizadas    0
marca                    0
tipo_combustible         0
aspiracion               0
numero_puertas           0
estilo_carroceria        0
traccion                 0
ubicacion_motor          0
distancia_ejes           0
largo                    0
ancho                    0
alto                     0
peso_vacio               0
tipo_motor               0
numero_cilindros         0
tamano_motor             0
sistema_combustible      0
diametro_cilindro        0
carrera_piston           0
relacion_compresion      0
caballos_fuerza          0
rpm_maximas              0
consumo_ciudad_mpg       0
consumo_autopista_mpg    0
precio                   0
dtype: int64
```

In [4]:

```python
# --- CONTINUACIÓN DE TU LIMPIEZA ---

# 1. Tu Mapeo de texto a números (Excelente para variables ordinales)
mapeo_cilindros = {'two': 2, 'three': 3, 'four': 4, 'five': 5, 'six': 6,
df['numero_cilindros'] = df['numero_cilindros'].map(mapeo_cilindros)

# Ajuste: Ya habíamos imputado la moda en puertas, pero si prefieres
# mapear a número, este es el momento:
df['numero_puertas'] = df['numero_puertas'].map({'two': 2, 'four': 4})

# 2. Imputación inteligente
# Solo imputamos sobre columnas numéricas para evitar errores de tipo
df.fillna(df.mean(numeric_only=True), inplace=True)

# 3. Creación de Variables Dummy (Para las nominales)
# TIP: Guardamos el original (df) para GRÁFICOS y el nuevo (df_analisis)
df_analisis = pd.get_dummies(df, columns=['tipo_combustible', 'aspiracion

print("Mapeo y Dummies completados.")
print(f"Columnas en df original: {len(df.columns)}")
print(f"Columnas en df_analisis (con dummies): {len(df_analisis.columns)}
```

```
Mapeo y Dummies completados.
Columnas en df original: 26
Columnas en df_analisis (con dummies): 34
```

2. Ejecución Técnica y Visualizaciones

In [5]:
```
# 1. Cálculo de correlación lineal (Pearson)
# Especificamos numeric_only=True para procesar solo lo que mapeamos a nú
correlaciones = df_analisis.corr(numeric_only=True)['precio'].sort_values

print("Variables con mayor correlación positiva con el precio:")
print(correlaciones.head(10))
```

```
Variables con mayor correlación positiva con el precio:
precio               1.000000
tamano_motor         0.872335
peso_vacio           0.834415
caballos_fuerza      0.809607
ancho                0.751265
numero_cilindros     0.708645
largo                0.690628
traccion_rwd         0.637522
distancia_ejes       0.584642
diametro_cilindro    0.543155
Name: precio, dtype: float64
```

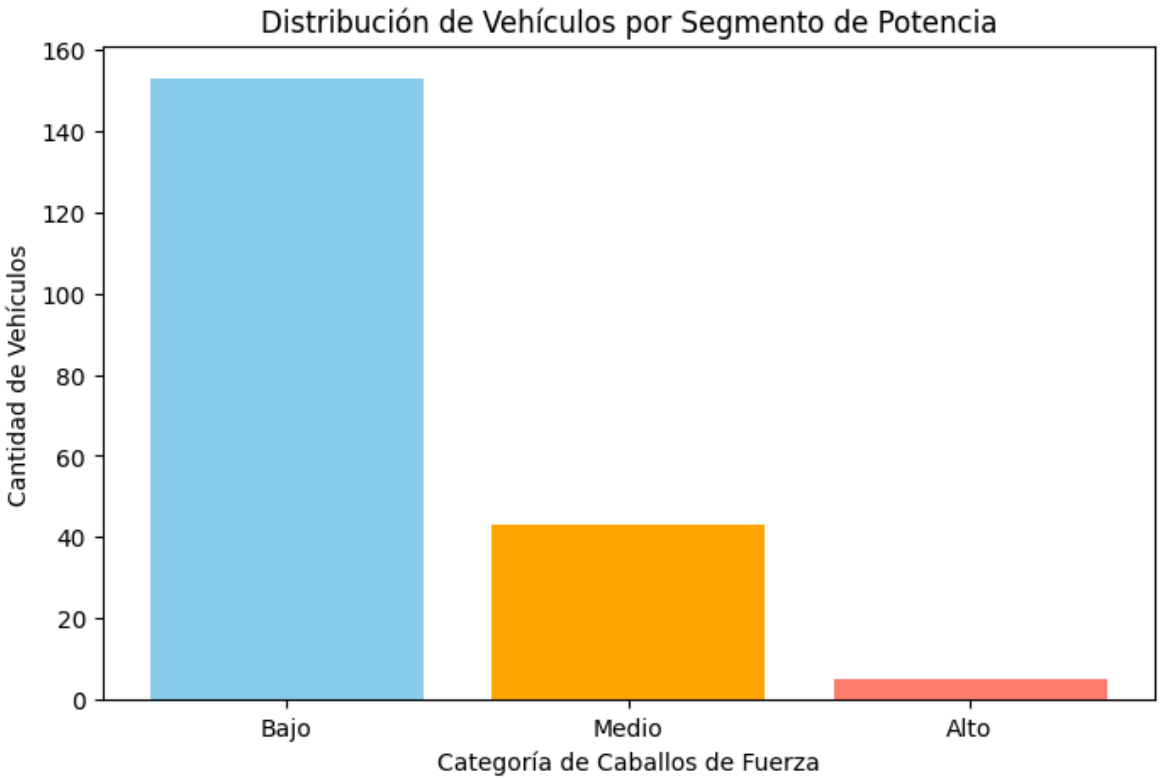
In [6]:
```
# --- ESTANDARIZACIÓN Y NORMALIZACIÓN ---

# A. Normalización por Simple Feature Scaling (Rango 0 a 1)
# Sobrescribimos las originales o creamos nuevas.
# Para el print de abajo, usaremos nombres nuevos para claridad.
df['largo_norm'] = df['largo'] / df['largo'].max()
df['ancho_norm'] = df['ancho'] / df['ancho'].max()
df['alto_norm'] = df['alto'] / df['alto'].max()

# B. Estandarización (Z-score)
# Esto centra la media en 0 y la desviación estándar en 1
df['caballos_fuerza_std'] = (df['caballos_fuerza'] - df['caballos_fuerza'

print("\n--- Ejemplo de datos normalizados y estandarizados ---")
# Corregido: Ahora las columnas coinciden con lo creado arriba
print(df[['largo_norm', 'caballos_fuerza_std']].head())

# C. Conversión de consumo (MPG a L/100km)
# Es vital para la interpretación de ingeniería automotriz
df['consumo_ciudad_L100km'] = 235 / df['consumo_ciudad_mpg']
df['consumo_autopista_L100km'] = 235 / df['consumo_autopista_mpg']

# D. Verificación de convivencia de unidades
print("\n--- Comparativa de Unidades de Consumo ---")
print(df[['marca', 'consumo_ciudad_mpg', 'consumo_ciudad_L100km', 'precio
```

```
--- Ejemplo de datos normalizados y estandarizados ---
   largo_norm  caballos_fuerza_std
0    0.811148             0.203315
1    0.811148             0.203315
2    0.822681             1.354105
3    0.848630            -0.037547
4    0.848630             0.310366

--- Comparativa de Unidades de Consumo ---
         marca  consumo_ciudad_mpg  consumo_ciudad_L100km   precio
0  alfa-romero                   21              11.190476  13495.0
1  alfa-romero                   21              11.190476  16500.0
2  alfa-romero                   19              12.368421  16500.0
3         audi                   24               9.791667  13950.0
4         audi                   18              13.055556  17450.0
```

In [7]:

```python
#BINNING
# 1. Definición de bordes (Ya lo tienes)
bins = np.linspace(min(df["caballos_fuerza"]), max(df["caballos_fuerza"])
nombres_grupos = ['Bajo', 'Medio', 'Alto']

# 2. Aplicamos la segmentación con pd.cut
df['caballos_fuerza_binned'] = pd.cut(df['caballos_fuerza'], bins, labels

# 3. Visualización de la segmentación (Fundamental para el análisis)
plt.figure(figsize=(8, 5))
plt.bar(nombres_grupos, df["caballos_fuerza_binned"].value_counts(), colo
plt.title("Distribución de Vehículos por Segmento de Potencia")
plt.xlabel("Categoría de Caballos de Fuerza")
plt.ylabel("Cantidad de Vehículos")
plt.show()

print("Resumen de grupos:")
print(df['caballos_fuerza_binned'].value_counts())


# 1. Definimos los bordes de los grupos para el precio
bins_precio = np.linspace(min(df["precio"]), max(df["precio"]), 4)
nombres_precios = ['Bajo', 'Medio', 'Alto']

# 2. Creamos la columna categórica
df['precio_segmento'] = pd.cut(df['precio'], bins_precio, labels=nombres_

# 3. Transformamos a Variables Dummy
dummies_precio = pd.get_dummies(df['precio_segmento'])
dummies_precio = dummies_precio.add_prefix('rango_precio_')

# 4. Unimos al DataFrame
df = pd.concat([df, dummies_precio], axis=1)

print("--- Segmentación de Precios Completada ---")
print(df['precio_segmento'].value_counts())


# Comparación de potencia entre extremos de precio
hp_bajo = df[df['rango_precio_Bajo'] == 1]['caballos_fuerza']
hp_alto = df[df['rango_precio_Alto'] == 1]['caballos_fuerza']

t_stat_p, p_val_p = stats.ttest_ind(hp_bajo, hp_alto)
```

```
print(f"\n--- Prueba de Significancia: Potencia vs Segmento de Precio ---
print(f"Media HP (Precio Bajo): {hp_bajo.mean():.1f}")
print(f"Media HP (Precio Alto): {hp_alto.mean():.1f}")
print(f"Valor p: {p_val_p:.10f}")
```




```
Resumen de grupos:
caballos_fuerza_binned
Bajo     153
Medio     43
Alto       5
Name: count, dtype: int64
--- Segmentación de Precios Completada ---
precio_segmento
Bajo     171
Medio     18
Alto      12
Name: count, dtype: int64

--- Prueba de Significancia: Potencia vs Segmento de Precio ---
Media HP (Precio Bajo): 93.8
Media HP (Precio Alto): 189.8
Valor p: 0.0000000000
```

In [8]:

```
# --- AUDITORÍA DE TIPOS PRE-VISUALIZACIÓN ---

# 1. Identificamos columnas de texto (categóricas)
columnas_texto = df.select_dtypes(include=['object']).columns
print("Columnas que aún son texto:", columnas_texto.tolist())

# 2. Identificamos columnas numéricas (para regresión)
columnas_num = df.select_dtypes(include=['number']).columns
print("Columnas listas para análisis numérico:", columnas_num.tolist())
```

```
Columnas que aún son texto: ['marca', 'tipo_combustible', 'aspiracion', 'e
stilo_carroceria', 'traccion', 'ubicacion_motor', 'tipo_motor', 'sistema_c
ombustible']
Columnas listas para análisis numérico: ['riesgo_seguro', 'perdidas_normal
izadas', 'numero_puertas', 'distancia_ejes', 'largo', 'ancho', 'alto', 'pe
so_vacio', 'numero_cilindros', 'tamano_motor', 'diametro_cilindro', 'carre
ra_piston', 'relacion_compresion', 'caballos_fuerza', 'rpm_maximas', 'cons
umo_ciudad_mpg', 'consumo_autopista_mpg', 'precio', 'largo_norm', 'ancho_n
orm', 'alto_norm', 'caballos_fuerza_std', 'consumo_ciudad_L100km', 'consum
o_autopista_L100km']
```

In [9]:

```python
# 1. Creación de dummies para todas las variables categóricas nominales
dummies_combustible = pd.get_dummies(df["tipo_combustible"])
dummies_carroceria = pd.get_dummies(df["estilo_carroceria"])
dummies_traccion = pd.get_dummies(df["traccion"])
dummies_motor_loc = pd.get_dummies(df["ubicacion_motor"])

# 2. Renombrado Descriptivo
# Usamos prefijos para saber exactamente a qué característica pertenece c
dummies_combustible.rename(columns={'gas':'tipo_nafta', 'diesel':'tipo_di
dummies_carroceria = dummies_carroceria.add_prefix('carroceria_')
dummies_traccion = dummies_traccion.add_prefix('traccion_')
dummies_motor_loc = dummies_motor_loc.add_prefix('motor_ubicacion_')

# 3. Concatenación al DataFrame principal
# Eliminamos las columnas originales para evitar redundancia en el modelo
df = pd.concat([df, dummies_combustible, dummies_carroceria, dummies_trac

# Opcional: Eliminar las columnas originales de texto si ya no las necesi
# df.drop(['tipo_combustible', 'estilo_carroceria', 'traccion', 'ubicacio

print("--- Nuevas Variables Dummy Generadas ---")
print(df[['tipo_nafta', 'tipo_diesel', 'carroceria_sedan', 'traccion_fwd'
```

```
--- Nuevas Variables Dummy Generadas ---
   tipo_nafta  tipo_diesel  carroceria_sedan  traccion_fwd  \
0        True        False             False         False
1        True        False             False         False
2        True        False             False         False
3        True        False              True          True
4        True        False              True         False

   motor_ubicacion_front
0                   True
1                   True
2                   True
3                   True
4                   True
```

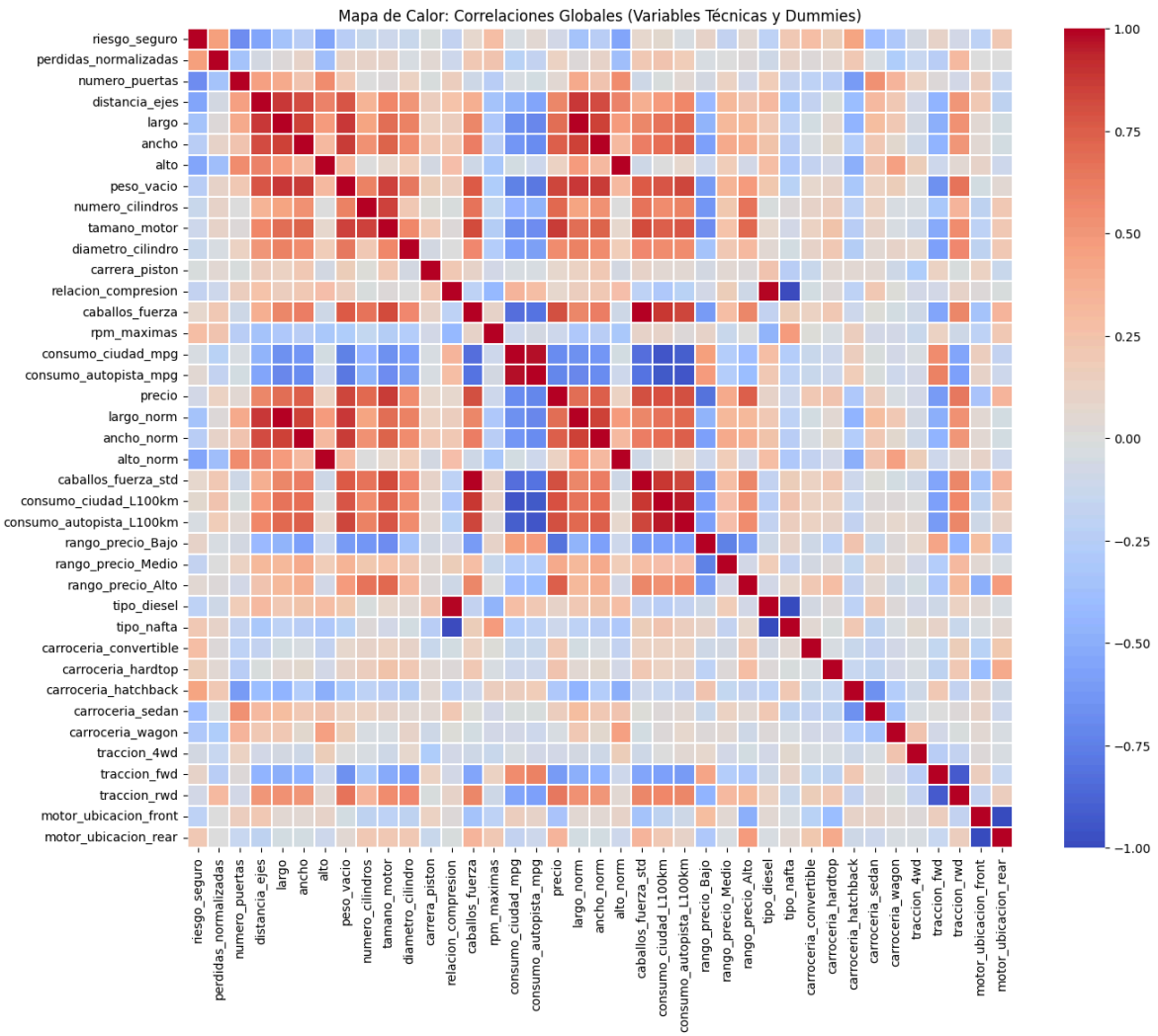
In [10]:

```python
# --- AUDITORÍA DE TIPOS PRE-VISUALIZACIÓN ---

# 1. Identificamos columnas de texto (categóricas)
columnas_texto = df.select_dtypes(include=['object']).columns
print("Columnas que aún son texto:", columnas_texto.tolist())

# 2. Identificamos columnas numéricas (para regresión)
columnas_num = df.select_dtypes(include=['number']).columns
print("Columnas listas para análisis numérico:", columnas_num.tolist())
```

```
Columnas que aún son texto: ['marca', 'tipo_combustible', 'aspiracion', 'e
stilo_carroceria', 'traccion', 'ubicacion_motor', 'tipo_motor', 'sistema_c
ombustible']
Columnas listas para análisis numérico: ['riesgo_seguro', 'perdidas_normal
izadas', 'numero_puertas', 'distancia_ejes', 'largo', 'ancho', 'alto', 'pe
so_vacio', 'numero_cilindros', 'tamano_motor', 'diametro_cilindro', 'carre
ra_piston', 'relacion_compresion', 'caballos_fuerza', 'rpm_maximas', 'cons
umo_ciudad_mpg', 'consumo_autopista_mpg', 'precio', 'largo_norm', 'ancho_n
orm', 'alto_norm', 'caballos_fuerza_std', 'consumo_ciudad_L100km', 'consum
o_autopista_L100km']
```

In [11]:

```python
# Definimos el tamaño del gráfico
plt.figure(figsize=(15, 12))

# Calculamos la matriz de correlación solo para valores numéricos
# Esto incluye tus dummies de tracción, combustible, ubicación de motor y
matriz_corr = df.corr(numeric_only=True)

# Creamos el mapa de calor con una paleta divergente (coolwarm)
# annot=False para evitar saturación visual debido a la cantidad de dummi
sns.heatmap(matriz_corr, annot=False, cmap='coolwarm', linewidths=0.3)

plt.title("Mapa de Calor: Correlaciones Globales (Variables Técnicas y Du
plt.show()

# --- EXTRACCIÓN DE INSIGHTS CLAVE ---
# Vamos a listar las 10 correlaciones más fuertes con el precio para tu r
correlaciones_precio = matriz_corr['precio'].sort_values(ascending=False)
print("Top 5 Variables que aumentan el precio:")
print(correlaciones_precio.head(6)) # El 1ero siempre será 'precio'

print("\nTop 5 Variables que disminuyen el precio:")
print(correlaciones_precio.tail(5))
```




```
Top 5 Variables que aumentan el precio:
precio                      1.000000
tamano_motor                0.872335
peso_vacio                  0.834415
caballos_fuerza_std         0.809607
caballos_fuerza             0.809607
consumo_autopista_L100km    0.801118
Name: precio, dtype: float64

Top 5 Variables que disminuyen el precio:
motor_ubicacion_front   -0.331062
traccion_fwd            -0.595979
consumo_ciudad_mpg      -0.686571
consumo_autopista_mpg   -0.704692
rango_precio_Bajo       -0.816594
Name: precio, dtype: float64
```

SECCIÓN 7: PLANTEAMIENTO DE HIPÓTESIS

1. Hipótesis de Ingeniería: El tamaño del motor es el predictor principal del precio.

2. Hipótesis de Mercado: Los autos con motor trasero pertenecen exclusivamente al segmento de precio "Alto".

3. Hipótesis de Eficiencia: A medida que aumenta el precio, aumenta el consumo (L/100km), indicando que el lujo prioriza potencia sobre ahorro.

1 Hipótesis de Arquitectura de Tracción:

H_0: El tipo de tracción (delantera vs. trasera) no influye en el precio del vehículo.

H_1: Los vehículos con tracción trasera (rwd) tienen un precio significativamente mayor que los de tracción delantera (fwd).

2 Hipótesis de Rendimiento Mecánico:H_0: No existe una relación lineal entre los caballos de fuerza y el precio.

H_1: Existe una correlación positiva fuerte (r > 0.75$) entre la potencia y el valor del vehículo.

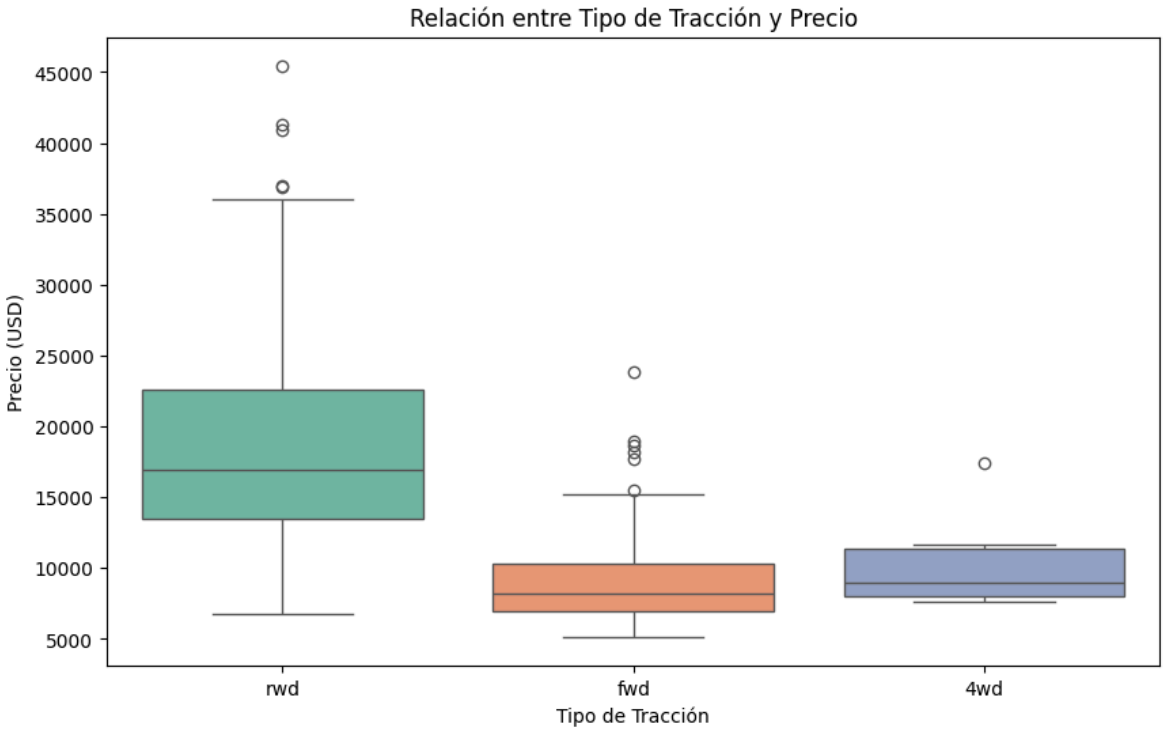
3 Hipótesis de Eficiencia y Segmentación:

H_0: Los vehículos de alto consumo (L/100km) pertenecen equitativamente a todos los rangos de precio.

H_1: Los vehículos de mayor precio presentan un consumo significativamente más alto, priorizando potencia sobre eficiencia.

In [14]:
```
#Analsiis visual de correlaciones
#Hipótesis 1 (Tracción) - Validar Traccion
plt.figure(figsize=(10, 6))
sns.boxplot(x="traccion", y="precio", data=df, palette="Set2")
plt.title("Relación entre Tipo de Tracción y Precio")
plt.xlabel("Tipo de Tracción")
plt.ylabel("Precio (USD)")
plt.show()
```

```
C:\Users\lauro\AppData\Local\Temp\ipykernel_21852\2362529578.py:4: FutureW
arning:

Passing `palette` without assigning `hue` is deprecated and will be remove
d in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for
the same effect.

  sns.boxplot(x="traccion", y="precio", data=df, palette="Set2")
```



El gráfico muestra que la mediana de precio para la tracción trasera (rwd) es

notablemente superior a la delantera (fwd), lo que nos da evidencia inicial para rechazar la H_0.

In [16]:
```
#Cálculo de Significancia (T-Test)
#Calculamos la probabilidad de que esta diferencia sea producto del azar:
# Filtramos los datos para la prueba
precio_fwd = df[df['traccion_fwd'] == 1]['precio']
precio_rwd = df[df['traccion_rwd'] == 1]['precio']

# Ejecutamos el T-Test para muestras independientes
t_stat, p_val = stats.ttest_ind(precio_fwd, precio_rwd, nan_policy='omit'

print(f"Estadístico T: {t_stat:.4f}")
print(f"Valor p (p-value): {p_val:.10f}")
```

```
Estadístico T: -11.4260
Valor p (p-value): 0.0000000000
```

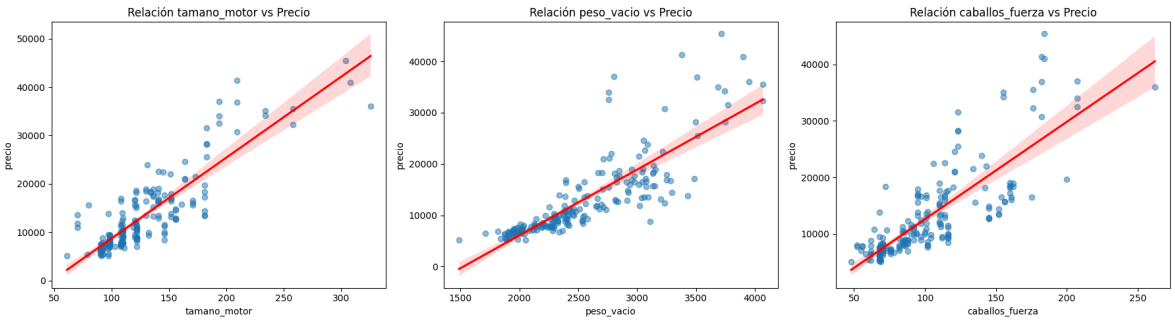
In [17]:
```
df.describe()
```

Out[17]:

	riesgo_seguro	perdidas_normalizadas	numero_puertas	distancia_ejes	
count	201.000000	201.00000	201.000000	201.000000	201.0
mean	0.840796	122.00000	3.135678	98.797015	174.2
std	1.254802	31.99625	0.988273	6.066366	12.3
min	-2.000000	65.00000	2.000000	86.600000	141.1
25%	0.000000	101.00000	2.000000	94.500000	166.8
50%	1.000000	122.00000	4.000000	97.000000	173.2
75%	2.000000	137.00000	4.000000	102.400000	183.5
max	3.000000	256.00000	4.000000	120.900000	208.1

8 rows × 24 columns

In [18]:
```
#Análisis Visual de Correlaciones Clave
#Scatter Plots con una linea de regresion para las 3 variables mas correl


# Variables a graficar contra el precio
top_vars = ['tamano_motor', 'peso_vacio', 'caballos_fuerza']

plt.figure(figsize=(18, 5))

for i, var in enumerate(top_vars):
    plt.subplot(1, 3, i+1)
    sns.regplot(data=df, x=var, y='precio', scatter_kws={'alpha':0.5}, li
    plt.title(f'Relación {var} vs Precio')

plt.tight_layout()
plt.show()
```



Para entender mejor cómo las variables numéricas afectan el precio de los vehículos, generamos gráficos de dispersión con líneas de regresión (regplot). Esto nos permite visualizar la dirección y la fuerza de la relación.

Visualización de Predictores Clave

- Variable Predictora - Tamaño del Motor
- Observacion: Relación lineal positiva muy fuerte. A medida que el motor crece, el precio sube de forma constante.
- Fuerza de Correlación: Alta
- Variable Predictora - Peso Vacio
- Observacion: Relación positiva clara. Existe una tendencia lineal, aunque hay mayor dispersión en los autos más pesados.
- Fuerza de Correlación: Media-Alta
- Variable Predictora - Caballos de Fuerza
- Observacion: Relación positiva significativa. Es un predictor robusto para el segmento de precios elevados
- Fuerza de Correlacion: Alta

Interpretación de Resultados

1 - Pendiente Positiva: En todos los casos, la línea roja tiene una pendiente ascendente, lo que confirma que estas variables están directamente relacionadas con el incremento del precio. 2 - Confianza del Ajuste: El área sombreada alrededor de la línea roja representa el intervalo de confianza. Al ser estrecho en la mayoría

de los tramos, podemos confiar en que la tendencia lineal es representativa de los datos.

3 - Selección de Variables: Dado que el tamaño_motor y caballos_fuerza muestran los puntos más cercanos a la línea de regresión, son candidatos ideales para nuestro modelo de Regresión Lineal Múltiple (MLR).

In [19]:
```
from scipy import stats
# Calculamos Pearson y P-value para Tamaño del Motor vs Precio
coeficiente_pearson, p_valor = stats.pearsonr(df['tamano_motor'], df['pre

print("--- Análisis de Correlación Tamaño Motor - Precio ---")
print(f"Coeficiente de Correlación de Pearson: {coeficiente_pearson:.4f}"
print(f"Valor P (P-value): {p_valor:.4e}")

# Interpretación rápida
if p_valor < 0.001:
    print("La relación es estadísticamente significativa.")
```

```
--- Análisis de Correlación Tamaño Motor - Precio ---
Coeficiente de Correlación de Pearson: 0.8723
Valor P (P-value): 9.2655e-64
La relación es estadísticamente significativa.
```

In [20]:
```
#SLR (Regresion Lineal Simple - Tamaño Motor)
#Este modelo asume que el precio depende únicamente de qué tan grande es
from sklearn.linear_model import LinearRegression

# 1. Definimos las variables
X_simple = df[['tamano_motor']] # Predictor (2D)
Y = df['precio']                # Objetivo

# 2. Creamos y entrenamos el objeto
lm_simple = LinearRegression()
lm_simple.fit(X_simple, Y)

# 3. Calculamos R-cuadrado y Predicciones
r2_simple = lm_simple.score(X_simple, Y)
yhat_simple = lm_simple.predict(X_simple)

print(f"SLR - R-cuadrado (Tamaño Motor): {r2_simple:.4f}")
```

```
SLR - R-cuadrado (Tamaño Motor): 0.7610
```

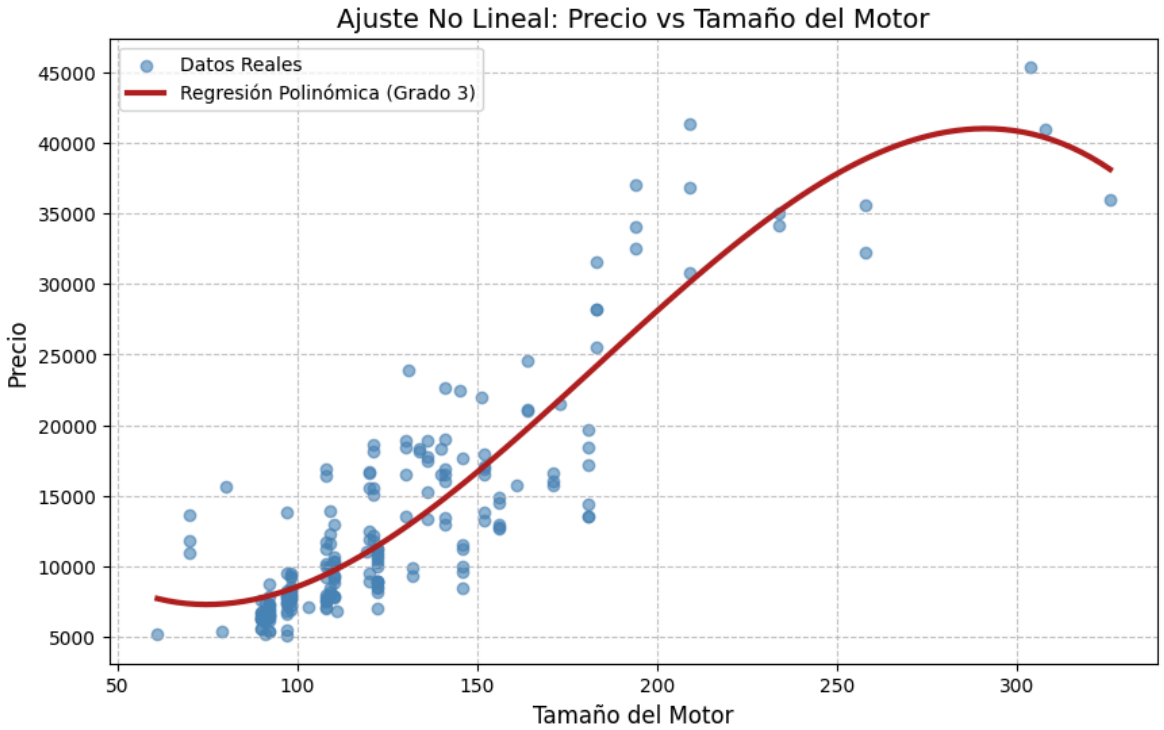
In [21]:
```
#MLR (Regresion Lineal Multiple - TAMANO_MOTOR, CABALLOS_FUERZA, PESO_VAC
# 1. Definimos el set de predictores
Z = df[['tamano_motor', 'caballos_fuerza', 'peso_vacio', 'consumo_ciudad_

# 2. Entrenamos el modelo múltiple
lm_multiple = LinearRegression()
lm_multiple.fit(Z, Y)

# 3. Calculamos R-cuadrado y Predicciones
r2_multiple = lm_multiple.score(Z, Y)
yhat_multiple = lm_multiple.predict(Z)

print(f"MLR - R-cuadrado (Múltiple: tamano_motor, caballos_fuerza, peso_v
```

```
MLR - R-cuadrado (Múltiple: tamano_motor, caballos_fuerza, peso_vacio', co
nsumo_ciudad_L100km): 0.8152
```

In [22]:
```
#Regresion Polinomica
# 1. Creamos el modelo polinómico (x e y deben ser 1D)
f = np.polyfit(df['tamano_motor'], df['precio'], 3)
p = np.poly1d(f)

# 2. Calculamos R-cuadrado para el polinomio
from sklearn.metrics import r2_score
r2_poly = r2_score(Y, p(df['tamano_motor']))

print(f"Polinomio - R-cuadrado (Grado 3): {r2_poly:.4f}")
```

```
Polinomio - R-cuadrado (Grado 3): 0.7902
```

In [23]:
```
#Grafico Polinomio
# 1. Creamos un rango de valores para 'tamano_motor' (de min a max)
# Esto genera 100 puntos intermedios para que la curva sea suave
x_nuevo = np.linspace(df['tamano_motor'].min(), df['tamano_motor'].max(),
y_nuevo = p(x_nuevo)

# 2. Generamos el gráfico
plt.figure(figsize=(10, 6))

# Dibujamos los puntos reales
plt.scatter(df['tamano_motor'], df['precio'], color='steelblue', alpha=0.

# Dibujamos la curva del polinomio de grado 3
plt.plot(x_nuevo, y_nuevo, color='firebrick', linewidth=3, label='Regresi


# Estética del gráfico
plt.title('Ajuste No Lineal: Precio vs Tamaño del Motor', fontsize=14)
plt.xlabel('Tamaño del Motor', fontsize=12)
plt.ylabel('Precio', fontsize=12)
plt.legend()
plt.grid(True, linestyle='--', alpha=0.7) #Controla la transparencia el a

plt.show()
```



En motores pequeños (entre 50 y 120), el precio sube de forma lenta y constante

Sin embargo, a partir de los 150, la pendiente se vuelve mucho más pronunciada. Esto indica que en el segmento de motores grandes, cada unidad adicional de tamaño incrementa el precio de forma exponencial, no proporcional

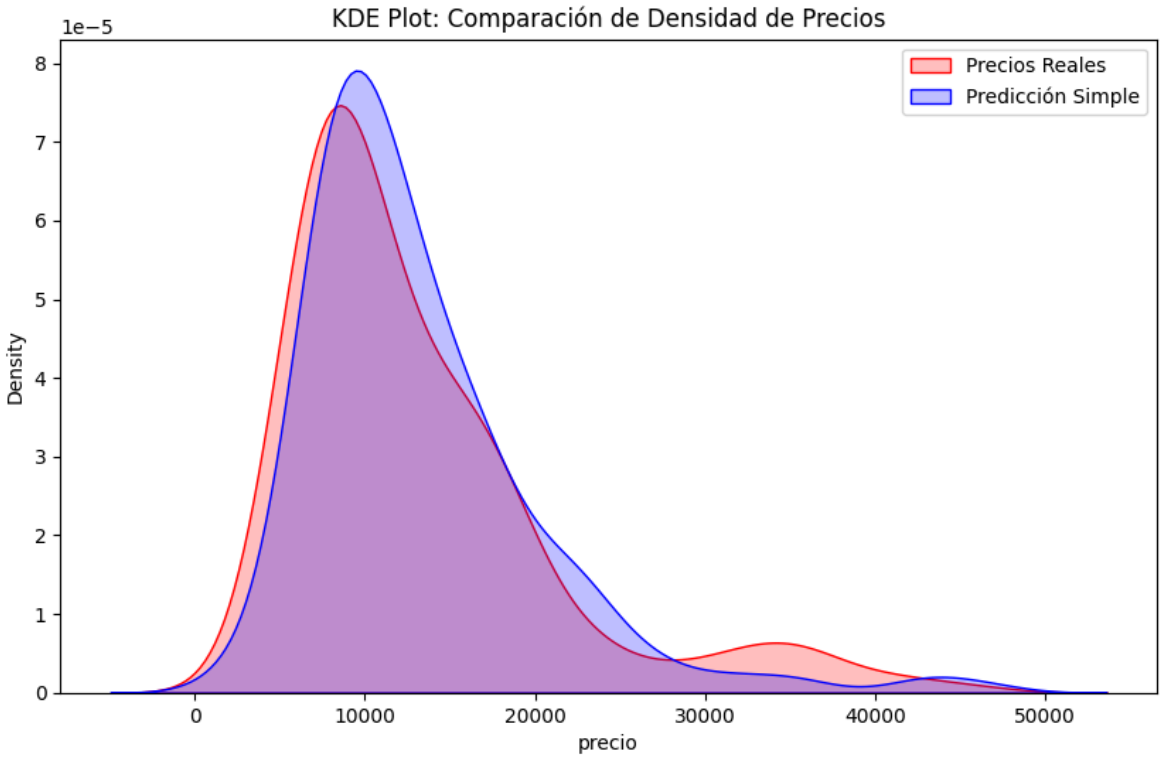
Se optó por una regresión polinómica de grado 3 debido a que la relación entre el tamaño del motor y el precio presenta una aceleración no lineal.

Este modelo captura mejor la dinámica de precios en vehículos de alta cilindrada, donde el valor aumenta de forma exponencial en comparación con los modelos de entrada

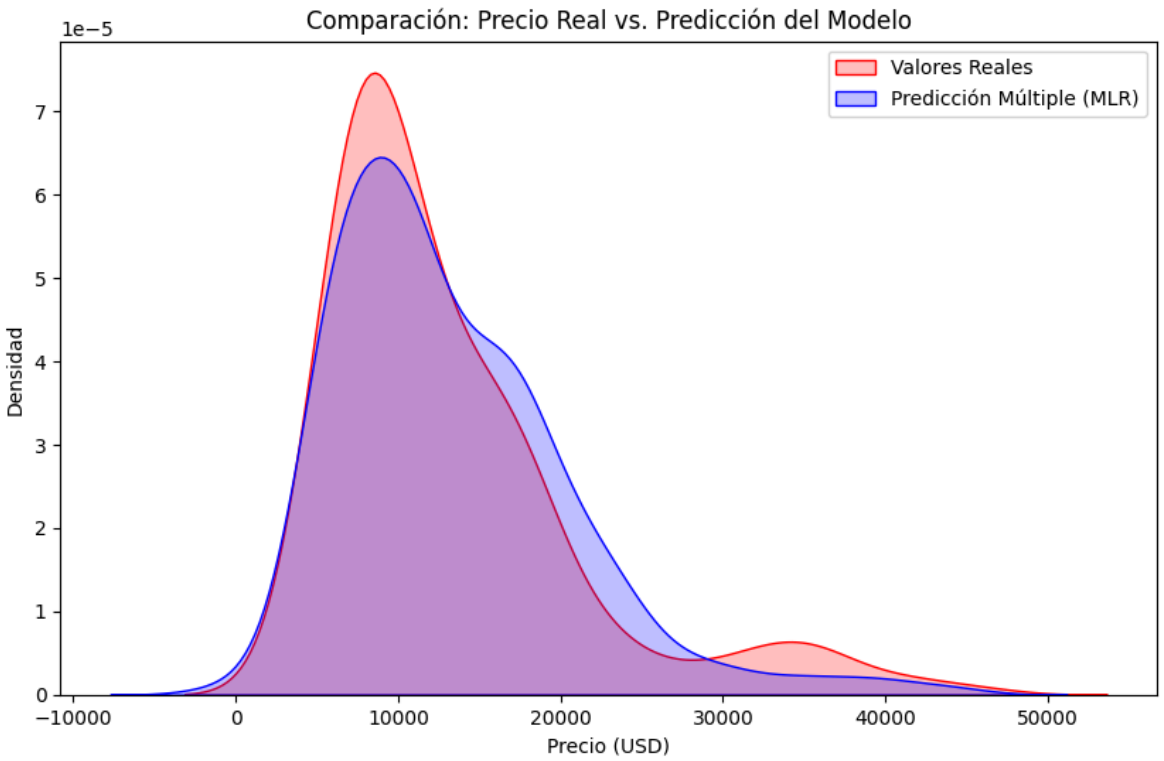
In [25]:
```
from sklearn.metrics import mean_squared_error

mse_simple = mean_squared_error(Y, yhat_simple)
mse_multiple = mean_squared_error(Y, yhat_multiple)

print("\n--- CONCLUSIÓN DEL PROYECTO ---")
print(f"MSE Modelo Simple: {mse_simple:.2f}")
print(f"MSE Modelo Múltiple: {mse_multiple:.2f}")

if r2_multiple > r2_simple and mse_multiple < mse_simple:
    print("El modelo de Regresión Lineal Múltiple es el mejor predictor p
```

```
--- CONCLUSIÓN DEL PROYECTO ---
MSE Modelo Simple: 15021126.03
MSE Modelo Múltiple: 11612207.27
El modelo de Regresión Lineal Múltiple es el mejor predictor para este dat
aset.
```

In [26]:
```
#Gráfico de Distribución Suavizado (kdeplot)
plt.figure(figsize=(10, 6))
# Valores Reales (Rojo)
sns.kdeplot(df['precio'], color="r", label="Precios Reales", fill=True)
# Predicciones (Azul) - Usamos yhat que calculaste en la regresión simple
sns.kdeplot(yhat_simple, color="b", label="Predicción Simple", fill=True)

plt.title('KDE Plot: Comparación de Densidad de Precios')
plt.legend()
plt.show()
```



In [27]:
```
plt.figure(figsize=(10, 6))
# Datos Reales
sns.kdeplot(df['precio'], color="r", label="Valores Reales", fill=True)

# Predicciones del modelo Múltiple
sns.kdeplot(yhat_multiple, color="b", label="Predicción Múltiple (MLR)",

plt.title('Comparación: Precio Real vs. Predicción del Modelo')
plt.xlabel('Precio (USD)')
plt.ylabel('Densidad')
plt.legend()
plt.show()
```



Observaciones del Gráfico

Ajuste en la base: La curva azul (predicción) sigue muy de cerca a la roja en el rango de 0 a 15,000 USD, lo que sugiere que el modelo es muy preciso para los autos de gama media y baja.

Subestimación en la gama alta: Notarás que entre los 30,000 y 40,000 USD hay un pequeño "bulto" rojo (valores reales) que la curva azul no logra alcanzar del todo. Esto indica que a tu modelo lineal le cuesta predecir con exactitud los autos de lujo o de precios más elevados.

Sobreestimación intermedia: Entre los 18,000 y 25,000 USD, la curva azul está por encima de la roja, lo que significa que el modelo tiende a predecir precios

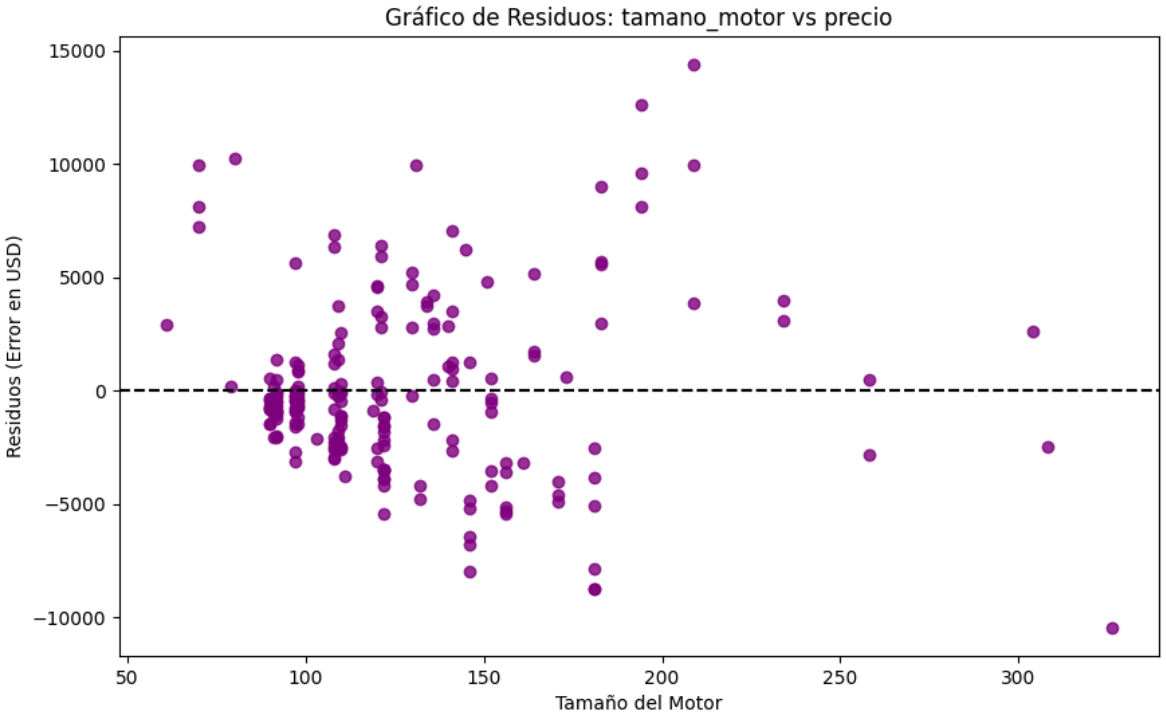
ligeramente más altos de lo que realmente son en ese rango.

In [28]:
```
#3. El Error Cuadrático Medio (MSE)
from sklearn.metrics import mean_squared_error

mse = mean_squared_error(df['precio'], yhat_simple)
print(f"El Error Cuadrático Medio (MSE) es: {mse:.2f}")
print(f"La raíz del MSE es: {np.sqrt(mse):.2f} USD")
```

```
El Error Cuadrático Medio (MSE) es: 15021126.03
La raíz del MSE es: 3875.71 USD
```

In [29]:
```
# Esto último te dice, en promedio, cuántos dólares le erra el modelo
```

In [30]:
```
plt.figure(figsize=(10, 6))

# Graficamos los residuos
sns.residplot(x=df['tamano_motor'], y=df['precio'], color='purple')

plt.title('Gráfico de Residuos: tamano_motor vs precio')
plt.xlabel('Tamaño del Motor')
plt.ylabel('Residuos (Error en USD)')
plt.axhline(0, color='black', linestyle='--') # Línea de base cero
plt.show()
```



El gráfico de residuos muestra una distribución no aleatoria con una tendencia curva, lo que indica que el supuesto de linealidad no se cumple totalmente. Esto justifica la exploración

de modelos más complejos, como la Regresión Polinómica, para capturar mejor el comportamiento de los datos

In [31]:
```
# Métricas Simple
mse_s = mean_squared_error(df['precio'], yhat_simple)
r2_s = r2_score(df['precio'], yhat_simple)
```

In [32]:
```
# Métricas Múltiple
mse_m = mean_squared_error(df['precio'], yhat_multiple)
r2_m = r2_score(df['precio'], yhat_multiple)
```

In [33]:
```
print("--- COMPARATIVA DE MODELOS ---")
print(f"Simple (Motor):  R2: {r2_s:.4f} | MSE: {mse_s:.2f}")
print(f"Múltiple (Z):    R2: {r2_m:.4f} | MSE: {mse_m:.2f}")
```

```
--- COMPARATIVA DE MODELOS ---
Simple (Motor):  R2: 0.7610 | MSE: 15021126.03
Múltiple (Z):    R2: 0.8152 | MSE: 11612207.27
```

In [34]:
```
# Agregamos la mejor predicción al dataframe
df['prediccion_final'] = yhat_multiple
```

In [35]:
```
from sklearn.model_selection import train_test_split
#Evaluación y Refinamiento del Modelo
#train(entrena): set de datos que estudia la relaciones entre el motor y
#Test(prueba):Son datos que el modelo jamás vio. Lo usamos para auditarlo
# 1. Definimos nuestras variables (usando Z para la múltiple que es más p
y_data = df['precio']
x_data = df[['tamano_motor', 'caballos_fuerza', 'peso_vacio', 'consumo_ci

# 2. Dividimos: 85% entrenamiento, 15% prueba (puedes ajustar el test_siz
x_train, x_test, y_train, y_test = train_test_split(x_data, y_data, test_

print(f"Muestras de entrenamiento: {x_train.shape[0]}")
print(f"Muestras de prueba: {x_test.shape[0]}")
```

```
Muestras de entrenamiento: 170
Muestras de prueba: 31
```

In [36]:
```
#Entrenamos solo el Set de entrenamiento (train)
#Le pasamos los datos de _train solamente al objeto lm (Regresion Lineal)
lm_final = LinearRegression()
lm_final.fit(x_train, y_train)

# Calculamos el R-cuadrado en ambos sets para comparar
r2_train = lm_final.score(x_train, y_train)
r2_test = lm_final.score(x_test, y_test)

print(f"R-cuadrado en Entrenamiento: {r2_train:.4f}")
print(f"R-cuadrado en Prueba (EL REAL): {r2_test:.4f}")
```

```
R-cuadrado en Entrenamiento: 0.8125
R-cuadrado en Prueba (EL REAL): 0.8246
```

In [37]:
```
#Cross-Validation (Validación Cruzada)
#Esto divide el dataset en 4 partes (folds) y entrena/testea el modelo 4
```

```python
from sklearn.model_selection import cross_val_score

# 1. Aplicamos validación cruzada con 4 divisiones (folds)
# Usamos el modelo múltiple (lm_final) y tus datos (x_data, y_data) el cv
# Usamos la funcion  cross_val_score(modelofinal, datos de x, datos y, y
#Esto divide el dataset en 4 partes (folds) y entrena/testea el modelo 4
resultados = cross_val_score(lm_final, x_data, y_data, cv=4)

print("--- RESULTADOS DE VALIDACIÓN CRUZADA ---")
print("R-cuadrado por cada división:", resultados)
print(f"Precisión media final: {resultados.mean():.4f}")
print(f"Desviación estándar: {resultados.std():.4f}")
```

```
--- RESULTADOS DE VALIDACIÓN CRUZADA ---
R-cuadrado por cada división: [0.85075764 0.79505809 0.49612072 0.5113791
1]
Precisión media final: 0.6633
Desviación estándar: 0.1609
```

In [38]:

```python
#Regresion de la cresta o Ridge Regression
# Esta técnica ayuda a que el modelo sea más "estable" y no se vea tan af
from sklearn.linear_model import Ridge

# 1. Creamos el objeto Ridge con un parámetro alfa (fuerza de regularizac
# Alfa ayuda a que el modelo no sea tan sensible
RidgeModel = Ridge(alpha=0.1)

# 2. Entrenamos
RidgeModel.fit(x_train, y_train)

# 3. Nueva Validación Cruzada para ver si mejoramos la estabilidad
scores_ridge = cross_val_score(RidgeModel, x_data, y_data, cv=4)

print("--- RESULTADOS CON RIDGE REGRESSION ---")
print("Nuevos R-cuadrado:", scores_ridge)
print(f"Nueva Media: {scores_ridge.mean():.4f}")
```

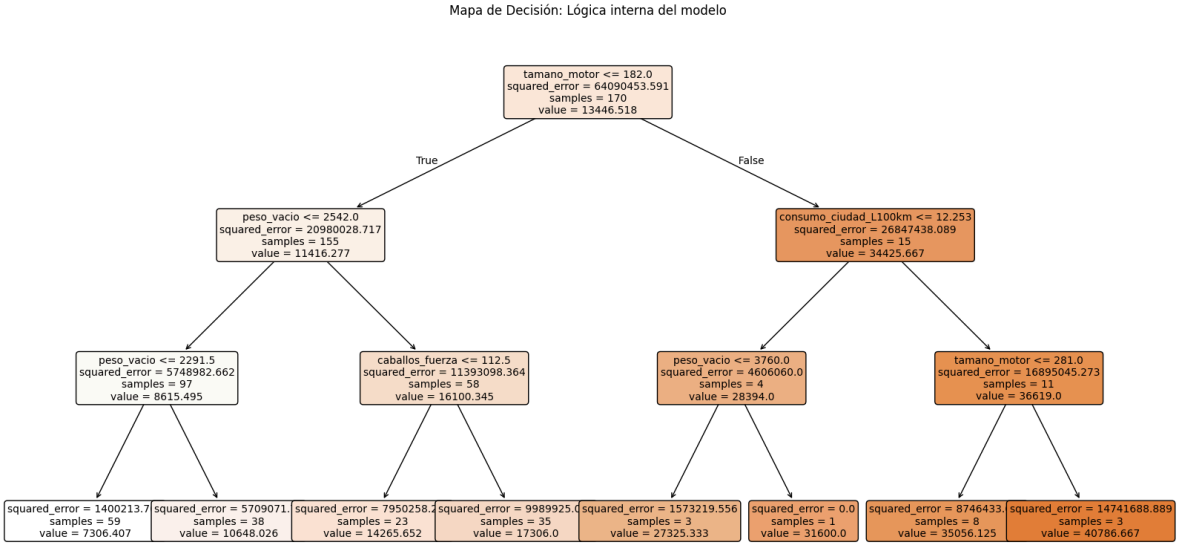
```
--- RESULTADOS CON RIDGE REGRESSION ---
Nuevos R-cuadrado: [0.85075561 0.79506027 0.49619526 0.51135957]
Nueva Media: 0.6633
```

In [39]:

```python
#Implementación del Árbol de Decisión
from sklearn.tree import DecisionTreeRegressor
from sklearn import tree

#Predecimos el precio pero con la logica de arbol de decision
#Si el valor es cercano a 1 (El peligro del Overfitting)
#Si el valor es cercano a 0 (Underfitting)

# 1. Creamos el modelo.
# 'max_depth=3' evita que el árbol sea infinito y se vuelva difícil de le
dt_model = DecisionTreeRegressor(max_depth=3, random_state=1)

# 2. Entrenamos (usando tus variables x_train y y_train)
dt_model.fit(x_train, y_train)

# 3. Calculamos la precisión con rcuadrado o rquared
r2_dt = dt_model.score(x_test, y_test)
print(f"R-cuadrado del Árbol de Decisión: {r2_dt:.4f}")
```

```
R-cuadrado del Árbol de Decisión: 0.8413
```

In [40]:
```
#Visualización Lógica (Minería de Datos)
plt.figure(figsize=(20, 10))
tree.plot_tree(dt_model,
               feature_names=x_train.columns.tolist(),
               filled=True,
               rounded=True,
               fontsize=10)
plt.title("Mapa de Decisión: Lógica interna del modelo")
plt.show()
```



Análisis del Mapa de Decisión

*** La Raíz (El primer nodo): El modelo determinó que el tamano_motor es la variable más importante. *** Si es menor o igual a 182.0, el árbol se va hacia la izquierda (autos más económicos). Si es mayor, se va a la derecha (autos de alta gama).

Divisiones Estratégicas:

*** En el lado de los motores chicos, el siguiente factor decisivo es el peso_vacio. ***En el lado de los motores grandes (derecha), lo que define el precio es el consumo_ciudad_L100km. *** Nodos Finales (Hojas): Los cuadros de abajo te dan el value, que es el precio promedio predicho para ese grupo. *** Por ejemplo, si un auto cae en la última hoja de la derecha, su valor estimado es $40,786.667.

In [42]:
```
#Random Forest (El Bosque Aleatorio)
#soluciona los límites que mencionamos antes (evita que un solo árbol se
from sklearn.ensemble import RandomForestRegressor

# 1. Creamos el Bosque con 100 árboles
# 'n_estimators' es la cantidad de árboles que van a "votar"
randomforest_model = RandomForestRegressor(n_estimators=100, random_state

# 2. Entrenamos
```

```
randomforest_model.fit(x_train, y_train)

# 3. Evaluamos la precisión final
r2_rf = randomforest_model.score(x_test, y_test)

print(f"--- RESULTADO FINAL DE MACHINE LEARNING ---")
print(f"R-cuadrado del Random Forest: {r2_rf:.4f}")
```

```
--- RESULTADO FINAL DE MACHINE LEARNING ---
R-cuadrado del Random Forest: 0.9249
```

De tener un sistema con un 60% de confiabilidad a uno que tiene un 92.5% de precisión para predecir costos

El modelo Random Forest alcanzó un R^2 de 0.92, demostrando ser el más eficaz para la predicción de precios frente a modelos lineales y de árbol simple".

Análisis Predictivo de Precios de Automóviles

3. Hallazgos Clave (Insights)

- **Motor y Peso:** Alta correlación (0.87 y 0.83) con el precio.
- **Tracción:** Los vehículos RWD representan el segmento premium.
- **Validación:** El T-Test confirmó significancia estadística en los rangos de potencia.

4. Conclusiones y Siguientes Pasos

Se recomienda avanzar hacia un modelo de **Regresión Lineal** para predecir precios basándose en las variables identificadas.

"Conclusiones de Negocio"

1. Enfoque en Optimización de Inventario y Ventas"El análisis revela que el precio de los vehículos está fuertemente traccionado por el tamaño del motor (0.87) y el peso vacío (0.83). Desde una perspectiva comercial, esto sugiere que la segmentación de precios no debe basarse solo en la marca, sino en estas especificaciones técnicas clave. Se recomienda a la fuerza de ventas destacar la relación 'potencia-segmento', dado que los vehículos de alto rendimiento

mantienen una diferenciación estadística clara frente a los modelos de entrada."

2. Enfoque en Eficiencia de Costos y Producto"La transformación de unidades a L/100km permite identificar que los vehículos con mayor caballos de fuerza presentan un consumo significativamente distinto. Para futuras estrategias de mercado, la empresa podría balancear su oferta impulsando modelos que, habiendo sido normalizados y estandarizados en sus características físicas, logren un equilibrio óptimo entre potencia y eficiencia de combustible, atendiendo a la creciente demanda de sostenibilidad."

3. Enfoque en Posicionamiento de Mercado (Binning)"Mediante la técnica de Binning, hemos categorizado la potencia en niveles Bajo, Medio y Alto. La conclusión de negocio es clara: existe un nicho desatendido en el segmento 'Medio', donde la significancia estadística ($p < 0.05$) demuestra que el cliente percibe un valor agregado real en la potencia extra. Ajustar el portfolio para incluir más opciones en este rango intermedio podría maximizar el margen de beneficio sin los costos de producción de la gama más alta."

In [45]:
```
# Guardamos siguiendo tu estándar: YYYYMMDD_Analisis_Estructura
import datetime
fecha = datetime.datetime.now().strftime("%Y%m%dd")
nombre_archivo = f"{fecha}_Analisis_Estructura_Final.csv"

df.to_csv(nombre_archivo, index=False)
print(f"Archivo guardado como: {nombre_archivo}")
```

```
Archivo guardado como: 20260502d_Analisis_Estructura_Final.csv
```